```
line = gets
newline = line.sub(/Python/, 'Ruby')       # replace first 'Python' with 'Ruby'
newerline = line.gsub(/Python/, 'Ruby') # replace every 'Python' with 'Ruby'
```

You can replace every occurrence of *JavaScript* and *Python* with *Ruby* using this:

```
line = gets
newline = line.gsub(/JavaScript|Python/, 'Ruby')
```

We'll have a lot more to say about regular expressions as we go through the book.

Blocks

This section briefly describes one of Ruby's particular strengths — *blocks*. A code block is a chunk of code you can pass to a method, as if the block were another parameter. This is an incredibly powerful feature, allowing Ruby methods to be extremely flexible. One of this book's early reviewers commented at this point: "This is pretty interesting and important, so if you weren't paying attention before, you should probably start now." We still agree.

Syntactically, code blocks are chunks of code that can be delimited one of two ways: between braces or between `do` and `end`. This is a code block at the end of a message call:

```
foo.each { puts "Hello" }
```

This is also a code block at the end of a message call:

```
foo.each do
  club.enroll(person)
  person.socialize
end
```

The two kinds of block delimiter have different precedence: the braces bind more tightly than the `do`/`end` pairs, a fact that will almost never make a difference in your code. In practice, the standard you will most often see is to use braces for single-line blocks and `do`/`end` for multiline blocks.

You can pass a block as an argument to any method call, even if the method doesn't do anything with the block. You do this by starting the block at the end of the method call, after any other parameters. For example, in the following code, the block containing `puts "Hi"` is associated with the call to the method `greet` (which we don't show here):

```
greet { puts "Hi" }
```

If the method has parameters, they appear before the block, and you can only pass one block per method call. In Blocks and Enumeration, on page 62, we'll see other ways to manage blocks and arbitrary chunks of code.

```
verbose_greet("Dave", "loyal customer") { puts "Hi" }
```

A method can then invoke an associated block one or more times using the Ruby `yield` statement. The `yield` statement invokes the block that was passed to the method, passing control to the code inside the block.

The following example shows a block call in action. We define a method that calls `yield` twice. We then call this method, putting a block on the same line after the call, and after any arguments to the method. You can think of the association of a block with a method as a kind of